

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск подстроки в строке

Студент гр. 4343

Семенчик В.В.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Выданное задание.

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T .

Вход:

- Первая строка — P
- Вторая строка — T

Выход:

индексы начал вхождений P в T , разделённые запятой; если P не входит в T , то вывести -1.

Тестовые данные			^
№ Теста	Входные данные	Выходные данные	
1	ab abab	0, 2	

Рисунок 1. Первая часть

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$).

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка — A
- Вторая строка — B

Выход:

Если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Тестовые данные			^
№ Теста	Входные данные	Выходные данные	
1	defabc abcdef	3	

Рисунок 2. Вторая часть

Кнут-Моррис-Пратт. Описание Алгоритма

Алгоритм состоит из двух этапов. На первом этапе для шаблона P строится префикс-функция. Создаётся вектор pi длиной m (где m - длина P), в котором $pi[i]$ хранит длину наибольшего собственного префикса подстроки $P[0..i]$, который одновременно является её суффиксом. Вычисление выполняется слева направо: для каждого i от 1 до $m-1$ пока j больше нуля и символ $P[i]$ не равен $P[j]$, j пересчитывается как $pi[j-1]$, после чего, если символы равны, j увеличивается на единицу, и $pi[i]$ присваивается текущее значение j . На втором этапе выполняется сам поиск. Переменная j (текущая длина совпавшего префикса) инициализируется нулём, затем для каждого символа текста $T[i]$ при j больше нуля и несовпадении $T[i]$ с $P[j]$ производится откат $j = pi[j-1]$. Если символы совпали, j увеличивается на единицу. В случае, когда j становится равным m , фиксируется вхождение с начальной позицией $i - m + 1$, после чего j снова устанавливается в $pi[j-1]$ для возможности обнаружения перекрывающихся вхождений. Для выявления циклического сдвига необходимо провести конкатенацию первой строки с самой собой. Если найдено хоть одно вхождение – то строка является циклическим сдвигом другой строки.

Исследование сложности.

Алгоритм имеет временную сложность $O(n+m)$: построение префикс-функции выполняется за $O(n)$, а поиск в тексте – за $O(m)$, где n -длина P , m -длина T .

Пространственная сложность – $O(m)$, что соответствует размеру pi (префикс-функция).

Выводы.

В ходе лабораторной работы был реализован алгоритм КМП и решены задачи на поиск вхождений подстроки в строку и выявление циклического сдвига.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.cpp

```
#include <iostream>
#include <string>
#include <vector>

#define LOG false
#define SHIFT false

using namespace std;

vector<int> buildPrefixFunction(const string& P) {
    int m = P.length();
    vector<int> pi(m, 0);

    if (LOG) cout << "Построение префикс-функции для шаблона: " << P <<
endl;

    int j = 0;
    for (int i = 1; i < m; i++) {
        if (LOG) cout << "   i=" << i << ", j=" << j << ", P[i]=" << P[i]
<< ", P[j]=" << P[j] << endl;

        while (j > 0 && P[i] != P[j]) {
            if (LOG) cout << "       несовпадение, j=" << j << " -> pi[" <<
j-1 << "]= " << pi[j-1] << endl;
            j = pi[j - 1];
        }

        if (P[i] == P[j]) {
            j++;
            if (LOG) cout << "       совпадение, j++ -> j=" << j << endl;
        }

        pi[i] = j;
        if (LOG) cout << "       pi[" << i << "]= " << j << endl;
    }

    if (LOG) {
        cout << "Готовая префикс-функция: ";
        for (int i = 0; i < m; i++) cout << pi[i] << " ";
        cout << endl;
    }

    return pi;
}

vector<int> KMPSearch(const string& P, const string& T) {
    int n = T.length();
    int m = P.length();

    if (m == 0) {
```

```

        if (LOG) cout << "Образец пустой, возврат пустого списка" <<
endl;
        return {};
    }

    if (LOG) cout << "Поиск образца \"" << P << "\" в тексте \"" << T <<
"\" << endl;

    vector<int> pi = buildPrefixFunction(P);
    vector<int> occurrences;

    int j = 0;

    for (int i = 0; i < n; i++) {
        if (LOG) cout << "i=" << i << ", T[i]=" << T[i] << ", j=" << j <<
", P[j]=" << (j < m ? string(1, P[j]) : "?") << endl;

        while (j > 0 && T[i] != P[j]) {
            if (LOG) cout << " несовпадение, откат j=" << j << " -> pi["
<< j-1 << "]" << pi[j-1] << endl;
            j = pi[j - 1];
        }

        if (T[i] == P[j]) {
            j++;
            if (LOG) cout << " совпадение, j++ -> j=" << j << endl;
        }

        if (j == m) {
            int pos = i - m + 1;
            occurrences.push_back(pos);
            if (LOG) cout << " найдено вхождение на позиции " << pos <<
endl;

            j = pi[j - 1];
            if (LOG) cout << " после нахождения, j=" << j << " (откат по
pi[" << m-1 << "]" << endl;
            if (SHIFT) {
                if (LOG) cout << " SHIFT режим, выход после первого
вхождения" << endl;
                return occurrences;
            }
        }
    }

    if (LOG) cout << "Поиск завершён, найдено вхождений: " <<
occurrences.size() << endl;

    return occurrences;
}

int main() {
    string P, T;

    cin >> P;
    cin >> T;

```

```

    if (LOG) cout << "Входные данные: образец=\"" << P << "\", текст=\""
<< T << "\", SHIFT=" << SHIFT << endl;

    vector<int> occurrences;

    if (SHIFT) {
        if (LOG) cout << "Режим выявления циклического сдвига" << endl;
        if (P.length() != T.length()) {
            if (LOG) cout << "Длины образца и текста не совпадают" <<
endl;

            cout << -1 << endl;
            return 0;
        }
        if (P.empty()) {
            if (LOG) cout << "Пустой образец" << endl;
            cout << 0 << endl;
            return 0;
        }
        string doubleP = P + P;
        if (LOG) cout << "Склеенный образец: " << doubleP << endl;
        occurrences = KMPSearch(T, doubleP);
    } else {
        if (LOG) cout << "Режим обычного поиска" << endl;
        occurrences = KMPSearch(P, T);
    }

    if (occurrences.empty()) {
        if (LOG) cout << "Результат: вхождений нет" << endl;
        cout << -1 << endl;
    } else {
        if (LOG) cout << "Результат: ";
        for (size_t i = 0; i < occurrences.size(); i++) {
            if (i > 0) cout << ",";
            cout << occurrences[i];
        }
        cout << endl;
    }

    return 0;
}

```


ПРИЛОЖЕНИЕ Б

ТЕСТИРОВАНИЕ ПРОГРАММЫ

Входные данные	Выходные данные
SHIFT=false	
ababac ababacab	0
aba abababacaba	0,2,4,8
xyz abcdefgh	-1
aaa aaaaaa	0,1,2,3
SHIFT=true	
abcd cdab	2
abcde abcde	0
abcd abdc	-1
aaaa aaaa	0
abc abcd	-1